

Image Recognition



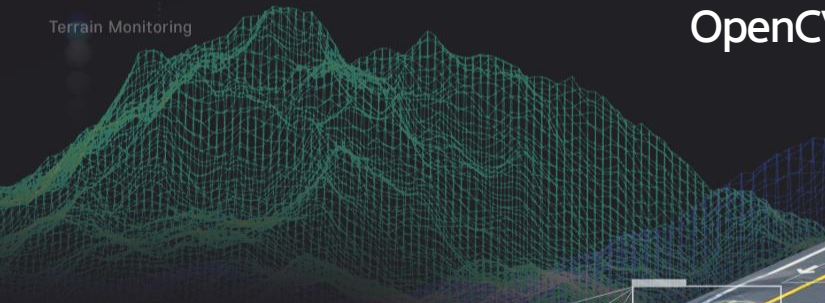
강원혁신플랫폼

컴퓨터 비전



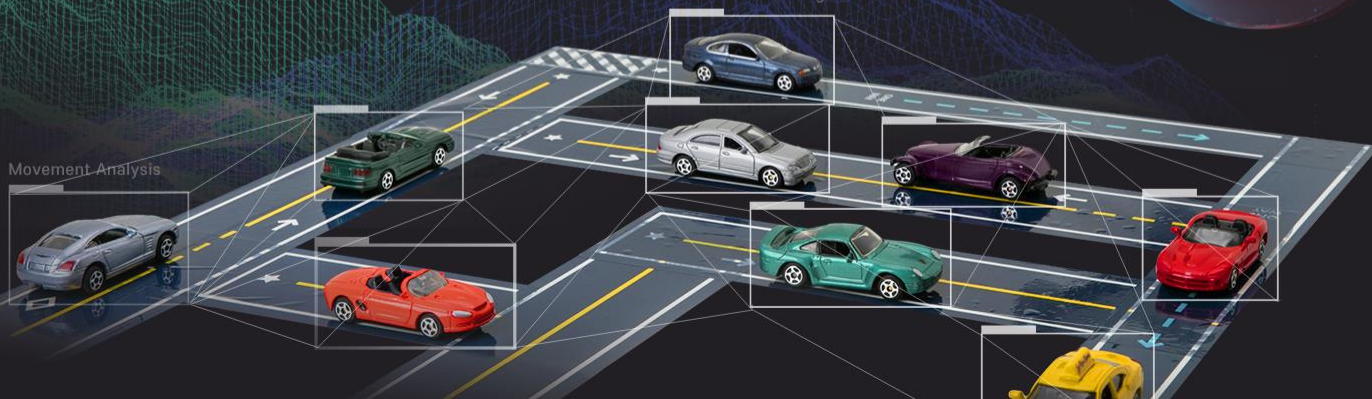
OpenCV로 영상 데이터 다루기

Terrain Monitoring



Autonomous Driving

Movement Analysis



이미지에 마스크 씌우기

cv2 모듈의 `bitwise_and()`, `bitwise_or()`, `bitwise_xor()` 함수를 이용하면 이미지를 비트 단위로 연산할 수 있음

- ▶ 이들 함수는 두 이미지를 비교하여 동일 위치에 있는 픽셀들을 비트 단위로 조작
 - ➡ 비트 단위 `and` 연산 : 두 이미지의 대응되는 비트들이 모두 1인 경우에 1이 되고, 그렇지 않으면 0이 됨
 - ➡ 비트 단위 `or` 연산 : 둘 중에서 하나만 1인 경우에는 1이 됨
 - ➡ 비트 단위 `not` 연산 : 하나의 이미지에 대해 적용하며 이 이미지의 비트들이 0은 1로, 1은 0으로 바뀌게 됨
 - ➡ 비트 단위 `xor` 연산 : 두 이미지의 대응 비트 중에 하나만 1인 경우에 1이 됨

이미지에 마스크 씌우기

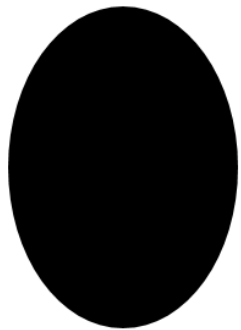
cv2 모듈의 `bitwise_and()`, `bitwise_or()`, `bitwise_xor()` 함수를 이용하면 이미지를 비트 단위로 연산할 수 있음

2 ▶ 비트 연산은 여러 가지 방식으로 활용

- ➡ 마스크(Mask)를 이용하여 이미지의 특정한 부분만 남기거나 사라지게 할 수 있음

이미지의 특정한 부분만 남기거나 사라지게 하는 동작을 확인해 보자.

흑백 이미지(mask_circle.png) 하나와 일반적인 이미지(iceberg.png) 하나를 준비



마스크 이미지



일반 이미지

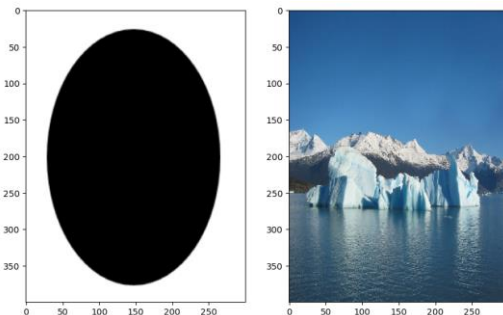
- ➡ 마스크로 사용할 이미지의 내부는 검정색으로 표현되어 있고 배경은 흰색임
- ➡ 따라서, 내부의 색상성분은 모두 0이고, 배경은 모두 1로 설정되었다고 이해하면 됨

이미지의 특정한 부분만 남기거나 사라지게 하는 동작을 확인해 보자.

다음은 마스크로 사용할 이미지(mask_circle.png)와 일반적인 이미지(iceberg.png)를 읽어와서 화면에 출력하는 코드

```
import cv2, os
import matplotlib.pyplot as plt

mask_image = cv2.imread(os.getcwd()+'/images/mask_circle.png', cv2.IMREAD_COLOR)
back_image = cv2.imread(os.getcwd()+'/images/iceberg.png', cv2.IMREAD_COLOR)
mask_image = cv2.resize(mask_image, (300,400)) # 이미지를 300x400 크기로 설정
back_image = cv2.resize(back_image, (300,400)) # 이미지를 300x400 크기로 설정
plt.figure(figsize=(10, 10))
plt.subplot(121)
plt.imshow(mask_image)
plt.subplot(122)
plt.imshow(back_image)
plt.show()
```



■ 비트단위로 **AND**, **OR**, **XOR** 연산자는 **&**, **|**, **^**를 사용하는데 그 의미는 다음과 같음

🔄 이러한 연산을 이미지에서 적용할 때 OpenCV는 `bitwise_and()`, `bitwise_or()`, `bitwise_xor()` 함수를 이용하여 처리함

연산자	의미	설명
&	비트 단위 AND	두 개의 피연산자의 해당 비트가 모두 1이면 1, 아니면 0
	비트 단위 OR	두 개의 피연산자의 해당 비트 중 하나라도 1이면 1, 아니면 0
^	비트 단위 XOR	두 개의 피연산자의 해당 비트의 값이 같으면 0, 아니면 1

AND

대응되는 비트가 1로 같으면 1

A	B	A & B
1	1	1
1	0	0
0	1	0
0	0	0

OR

대응되는 비트 중에서 하나라도 1이면 1

A	B	A B
1	1	1
1	0	1
0	1	1
0	0	0

XOR

대응되는 비트가 서로 다르면 1

A	B	A ^ B
1	1	0
1	0	1
0	1	1
0	0	0

NOT

비트를 반전함

A	~A
1	0
0	1

비트 AND 연산



- 두 이미지를 겹쳐 놓은 뒤에 동일한 위치의 비트가 모두 1인 경우만 1이 되고, 그렇지 않으면 0이 됨
 - 1 ➡ 마스크로 사용되는 이미지의 0은 배경 이미지가 무엇이든 관계없이 0으로 만드는 것
 - ⦿ 즉, 검은색 부분은 배경 이미지와 관계없이 0으로 만들어 버림
 - 2 ➡ 마스크로 사용되는 이미지의 1은 배경 이미지의 값이 해당 픽셀의 색을 결정
 - ⦿ 즉, 흰색 부분은 배경 이미지의 값이 해당 픽셀의 색을 결정
 - ⦿ 배경 이미지가 투명하게 통과하는 것을 의미

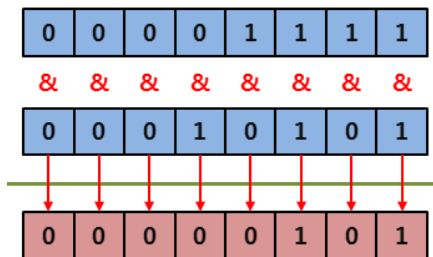
정리하면, 마스크(마스크로 사용되는 이미지의 0)의 검은색 부분으로 배경 이미지를 가리는 것

비트 AND 연산

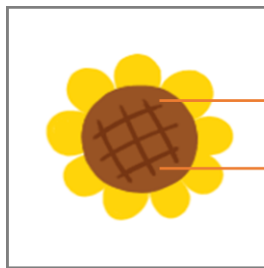


- 대응되는 두 비트가 모두 1일 때만 1을 반환하며, 다른 경우는 모두 0을 반환

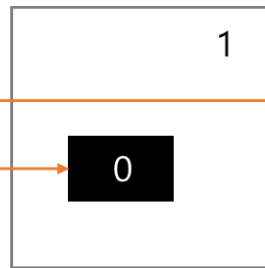
비트 AND 연산



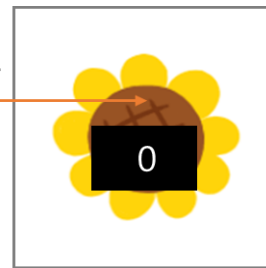
flag	0000	0111	비트 AND 연산 ↓
~mask	1111	1101	
flag	0000	0101	



배경 이미지



마스크



비트 단위 AND

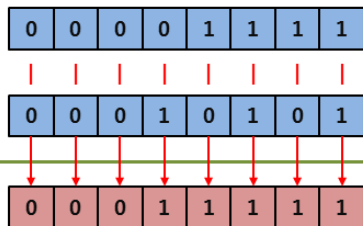
비트 OR 연산



■ 마스크의 1인 부분에 해당되는 픽셀은 무조건 1이 되고, 마스크의 0인 부분은 배경 이미지의 픽셀에 따라 결정

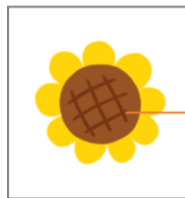
🔄 따라서, 흰색(마스크의 1) 부분이 그림을 가리는 마스크의 역할을 하는 것

비트 OR 연산

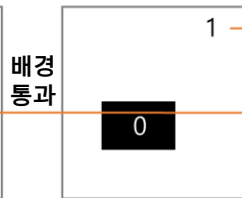


flag	0	0	0	0	0	0	0
mask	0	0	0	0	1	0	0
flag	0	0	0	0	0	1	0

비트 OR 연산

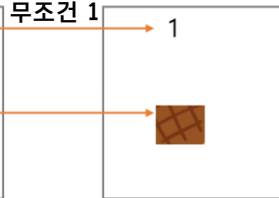


배경 이미지



배경 통과

마스크



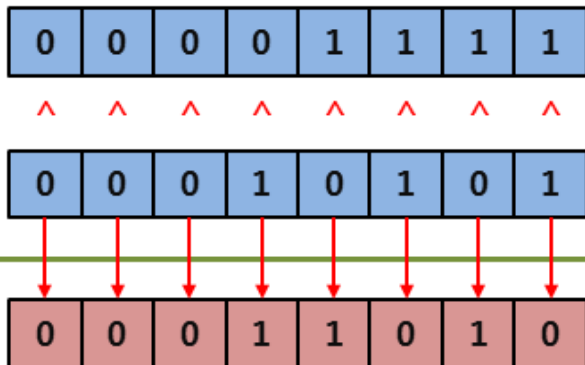
비트 단위 OR

비트 XOR 연산



- XOR 연산의 성질을 생각하면 0인 부분은 원래 이미지의 값이 그대로 유지되고, 1인 부분은 비트를 반대로 뒤집히게 되는 것을 이해할 수 있을 것임

비트 XOR 연산

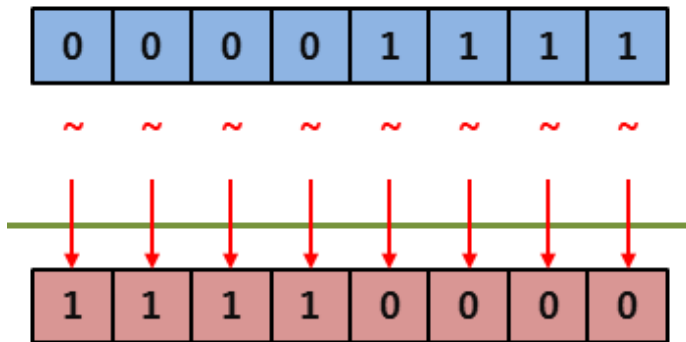


비트 NOT 연산



- 비트 NOT 연산자는 해당 비트가 1이면 0을 반환하고, 0이면 1을 반환
- 🔄 비트 NOT 연산자(~)의 동작을 나타냄

비트 NOT 연산



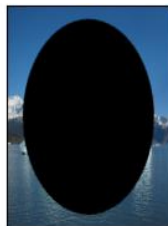
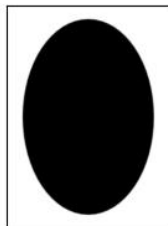
다음은 OpenCV의 bitwise_and(), bitwise_or(), bitwise_xor() 함수로 마스크와 비트연산(and, or, xor)을 수행한 결과를 이미지로 생성하는 코드이다.

```
import cv2, os

mask_image = cv2.imread(os.getcwd()+'/images/mask_circle.png', cv2.IMREAD_COLOR)
back_image = cv2.imread(os.getcwd()+'/images/iceberg.png', cv2.IMREAD_COLOR)
mask_image = cv2.resize(img1, (300,400))          # 이미지를 300x400 크기로 설정
back_image = cv2.resize(img2, (300,400))          # 이미지를 300x400 크기로 설정

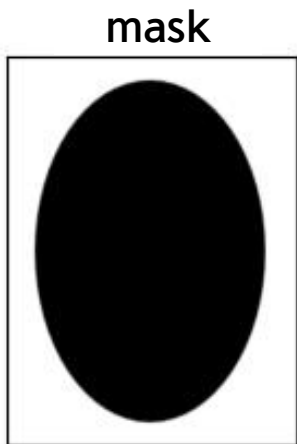
mask_ANDed = cv2.bitwise_and(mask_image, back_image) # 마스크와 비트단위 AND 연산
mask_ORed = cv2.bitwise_or(mask_image, back_image)  # 마스크와 비트단위 OR 연산
mask_XORed = cv2.bitwise_xor(mask_image, back_image) # 마스크와 비트단위 XOR 연산
```

마스크와 비트연산(and, or, xor)을
수행한 결과는 다음과 같다.



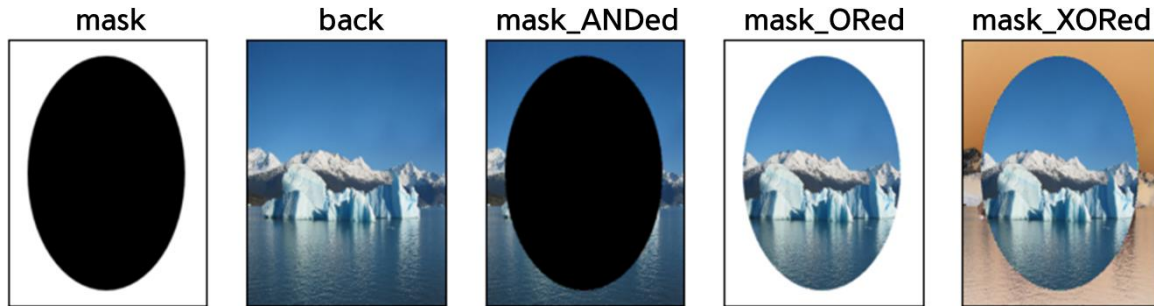
마스크와 비트연산(and, or, xor)을 수행한 결과는 다음과 같다.

마스크와 비트연산(and, or, xor)을 수행한 결과는 다음과 같다.



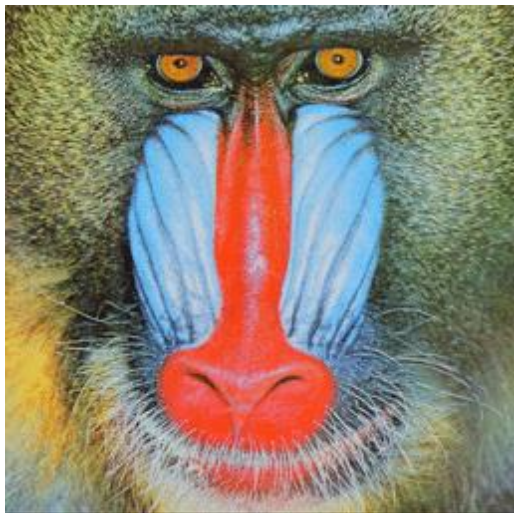
다음은 마스크와 비트연산을 수행한 결과를 화면에 출력하는 코드이다.

```
plt.figure(figsize=(10, 10))
plt.subplot(151)
plt.imshow(mask_image)
plt.title("mask")
plt.xticks([])
plt.yticks([])
plt.subplot(152)
plt.imshow(back_image)
plt.title("back")
plt.xticks([])
plt.yticks([])
plt.subplot(153)
plt.imshow(mask_ANDed)
plt.title("mask_ANDed")
plt.xticks([])
plt.yticks([])
plt.subplot(154)
plt.imshow(mask_ORed)
plt.title("mask_ORed")
plt.xticks([])
plt.yticks([])
plt.subplot(155)
plt.imshow(mask_XORed)
plt.title("mask_XORed")
plt.xticks([])
plt.yticks([])
plt.show()
```

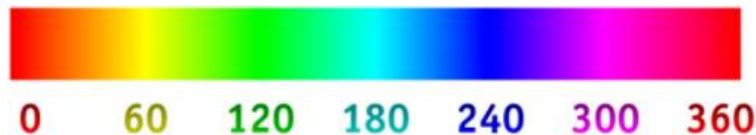


OpenCV를 이용하여 이미지에서 특정한 색상의 데이터를 추출해 보자.

아래의 맨드릴 원숭이 이미지에서
푸른색 픽셀들만 추출



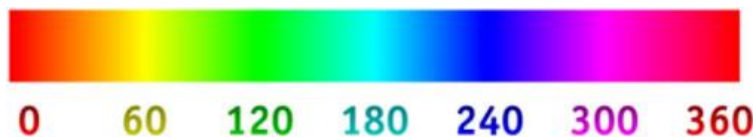
푸른색 픽셀만 추출할 때에는 RGB나 BGR 방식의
색상 모드보다는 HSV 방식이 더 적합



HSV 방식

OpenCV를 이용하여 이미지에서 특정한 색상의 데이터를 추출해 보자.

HSV 방식에서 색상 범위를 0도에서 360도로 잡을 때에
푸른색은 대략 190도에서 260도 정도의 범위를 차지하고 있음



HSV 방식

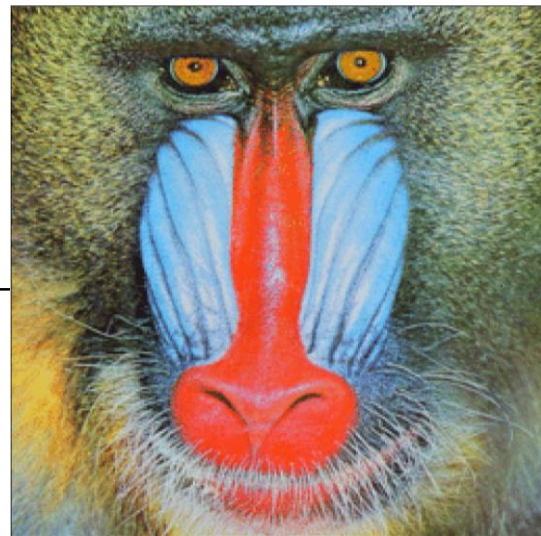
- ☞ 따라서, 이미지에서 푸른색 픽셀만 추출하고 싶다면 HSV로 변경한 뒤에 H 값이 190도에서 260도 사이에 있는 픽셀만 추출하면 될 것임
- ☞ 이미지에서 색상의 범위를 0에서 360 사이의 정수로 표현하면 한 바이트 이상이 필요함
- ☞ 그래서 이미지 파일에서는 이 값을 $\frac{1}{2}$ 로 줄여 0도에서 180도 범위로 표현함
- ☞ 따라서 푸른색을 추출하려면 80도에서 130도 사이의 색상 값을 가진 픽셀을 찾으면 됨

다음은 맨드릴 원숭이 원본 이미지를 화면에 출력하는 코드이다.

```
import cv2, os
import matplotlib.pyplot as plt

image = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 10))
plt.imshow(image)
plt.title("맨드릴 원숭이")
plt.xticks([])
plt.yticks([])
plt.show()
```



다음은 맨드릴 원숭이 이미지에서 푸른색 픽셀만 추출하는 코드이다.

```
import cv2, os
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
image_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV) # HSV 모드로 변환

blue_low = np.array([80, 0, 0]) # 푸른색의 아래쪽 경계를 색상 80
blue_high = np.array([130, 255, 255]) # 푸른색의 위쪽 경계를 색상 130
blue_mask = cv2.inRange(image_hsv, blue_low, blue_high) # 푸른색 픽셀 선택
blue_mask_rgb = cv2.cvtColor(blue_mask, cv2.COLOR_BGR2RGB)
extracted = cv2.bitwise_and(image_rgb, image_rgb, mask=blue_mask) # cv2.bitwise_and 연산

plt.figure(figsize=(10, 10))
plt.subplot(131)
plt.imshow(image_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(132)
plt.imshow(blue_mask_rgb)
plt.title("blue_mask")
plt.xticks([])
plt.yticks([])
plt.subplot(133)
plt.imshow(extracted)
plt.title("extracted")
plt.xticks([])
plt.yticks([])
plt.show()
```

푸른색 추출 : 80도 ~ 130도 사이의 색상 값 추출

OpenCV의 `cvtColor()` 함수를 이용하여 OpenCV의 기본 이미지 모드인 BGR 모드로 되어 있는 이미지를 HSV 모드로 바꾼다.

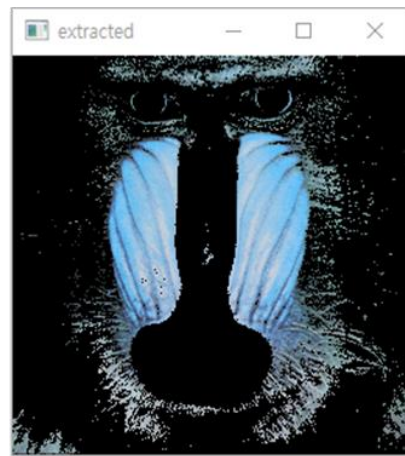
- 푸른색의 아래쪽 경계를 색상 80으로 잡고, 위쪽 경계를 색상 130으로 잡음
- 채도와 명도는 고려하지 않을 것이기에 아래쪽 한계를 0, 위쪽 한계를 255로 잡음



원본 이미지



푸른색 마스크



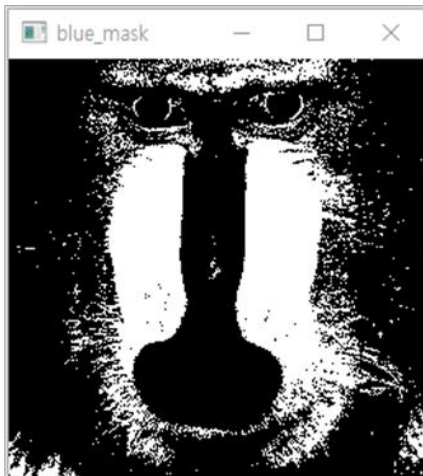
(원본 이미지)푸른색 픽셀만 추출한 이미지

OpenCV의 `inRange()` 함수를 이용하여 80~130도 범위에 있는 픽셀을 찾아낸다.

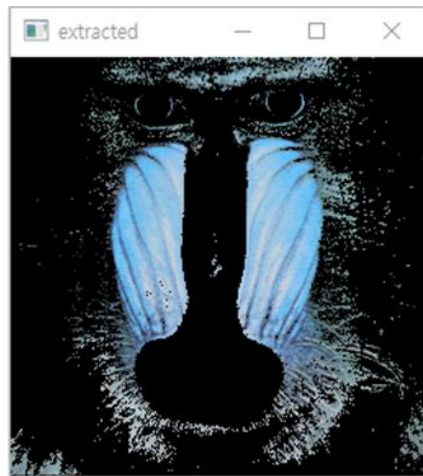
즉, 80~130도 범위에 있는 픽셀들은 1(흰색), 그렇지 않은 픽셀은 0(검은색)으로 표시한 이미지를 뽑아줌



원본 이미지



푸른색 마스크



(원본 이미지)푸른색 픽셀만 추출한 이미지

이미지 모드를 HSV 모드로 변경한 뒤에 H 값이 80도에서 130도 사이의 색 값을 가진 픽셀로 추출된 이미지와 원본 이미지를 비교해 보자.

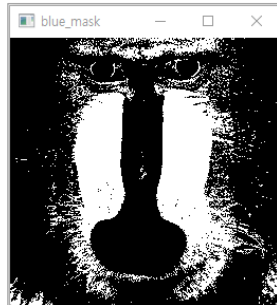
이미지를 HSV 모드로 변환한 `image_hsv`에서 `blue_low ~ blue_high` 범위에 있는 픽셀은 1(흰색), 그렇지 않은 픽셀은 0(검은색)으로 이미지를 뽑아냄

```
blue_low = np.array([80, 0, 0])      # 푸른색의 아래쪽 경계를 색상 80
blue_high = np.array([130, 255, 255]) # 푸른색의 위쪽 경계를 색상 130
blue_mask = cv2.inRange(image_hsv, blue_low, blue_high) # 푸른색 픽셀 선택
```

아래 그림의 오른쪽 흰색 부분은 원본 이미지에서 푸른색을 뺀 곳임



원본 이미지



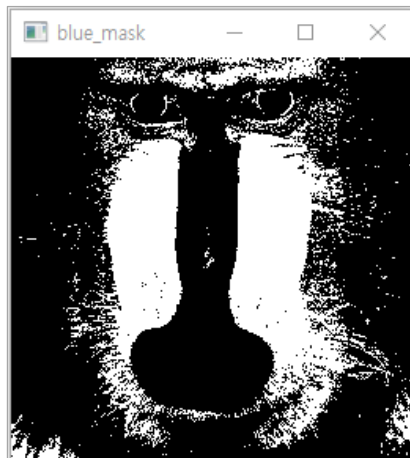
푸른색 마스크

이미지 모드를 HSV 모드로 변경한 뒤에 H 값이 80도에서 130도 사이의 색 값을 가진 픽셀로 추출된 이미지와 원본 이미지를 비교해 보자.

아래 그림의 **오른쪽 흰색 부분**은 원본 이미지에서 **푸른색**을 뺀 곳임



원본 이미지



푸른색 마스크

추출한 이미지를 마스크로 이용하여 원래의 이미지에서 푸른색 픽셀만을 골라낼 수 있다.

이때 주의할 점은 `cv2.bitwise_and(image_rgb, blue_mask)`라고 하면,
컬러 이미지인 `image_rgb`와 흑백 이미지인 `blue_mask`가 가진 채널의 수가 달라
배열의 크기가 다르기 때문에 오류가 발생

```
print(image_rgb.shape) # (256, 256, 3)
print(blue_mask.shape) # (256, 256)
extracted = cv2.bitwise_and(image_rgb, image_rgb, mask=blue_mask) # cv2.bitwise_and 연산
```

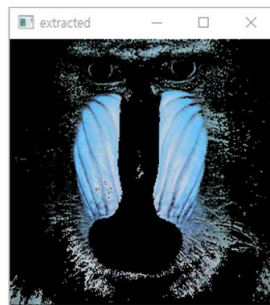
앞 코드에서 같은 이미지로 `and`를 하면서
특정 영역에서만 연산이 일어나도록
마스킹을 적용하는 방식을 사용한 것임



원본 이미지



푸른색 마스크



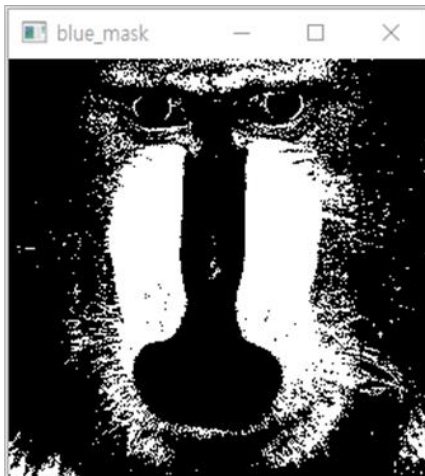
푸른색 픽셀만
추출한 이미지

추출한 이미지를 마스크로 이용하여 원래의 이미지에서 푸른색 픽셀만을 골라낼 수 있다.

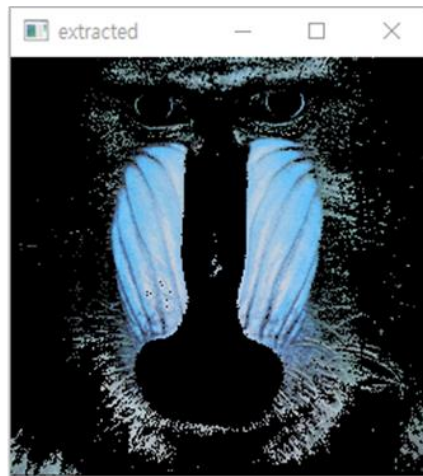
앞 코드에서 같은 이미지로 and를 하면서 특정 영역에서만 연산이 일어나도록 마스크를 적용하는 방식을 사용한 것임



원본 이미지



푸른색 마스크



(원본 이미지)푸른색 픽셀만 추출한 이미지